

Instituto Federal de Mato Grosso — IFMT

Disciplina: Transmissão e Comunicação de Dados

Projeto Final

Transmissão de Dados Confiável via Rádio

Sistema de comunicação sem fio com ESP32 e módulos RF 433 MHz

Alunos:

Pedro Henrique Rodrigues
Danilo Vinícius Galvão de Moraes

Professor: Ruy de Oliveira
Cuiabá — MT, 2026

1. Introdução

Este relatório descreve o desenvolvimento de um sistema de comunicação sem fio entre dois microcontroladores ESP32, utilizando módulos de radiofrequência (RF) de 433 MHz. O objetivo é implementar protocolos que garantam a transmissão confiável de mensagens de texto e imagens, aplicando conceitos da disciplina de Transmissão e Comunicação de Dados.

O sistema é composto por um transmissor, que envia os dados, e um receptor, que exibe as informações em um display LCD 16x2. A comunicação ocorre exclusivamente pelo canal sem fio RF, sem qualquer conexão física de dados entre os dois ESP32.

2. Materiais Utilizados

- 2 placas ESP32
- 2 pares de módulos RF 433 MHz (1 transmissor + 1 receptor em cada ESP32)
- 1 display LCD 16x2 com módulo I2C (endereço 0x27)
- Cabos jumpers e protoboard
- Cabo de cobre
- 1 computadores com Arduino IDE para programação e monitoramento serial

3. Arquitetura do Sistema

Cada ESP32 possui dois módulos RF: um transmissor e um receptor. Isso é necessário porque a comunicação é bidirecional — o ESP32 transmissor envia dados e recebe ACK; o ESP32 receptor recebe dados e envia ACK.

3.1 Pinos Utilizados

Componente	Pino ESP32	Função	Dispositivo
TX_PIN	27	Transmissão RF	Módulo TX
RX_PIN	26	Recepção RF	Módulo RX
TX_VCC_PIN	vin	Alimentação TX	Módulo TX
SDA	21	Dados I2C	LCD (receptor)
SCL	22	Clock I2C	LCD (receptor)

4. Codificação de Linha — 4B6B

Os módulos RF 433 MHz são sensíveis a longas sequências de bits iguais, o que causa perda de sincronização. Para resolver isso, foi implementada a codificação 4B6B: cada byte é dividido em dois nibbles (4 bits), e cada nibble é mapeado para um símbolo de 6 bits com pelo menos dois bits '1' e dois bits '0', garantindo transições frequentes no sinal.

Exemplo com o byte 0x41 (letra A):

```
Nibble alto: 0x4 => símbolo 0x16 (010110)
Nibble baixo: 0x1 => símbolo 0x0E (001110)
Transmite: 0x16 0x0E (em vez de 0x41)
```

5. Estrutura do Quadro

Cada unidade de dados transmitida é um quadro (frame) de 21 bytes fixos:

FLAG (1B)	TYPE (1B)	SEQ (1B)	LEN (1B)	DATA (16B)	FCS (1B)
-----------	-----------	----------	----------	------------	----------

- FLAG (0x7E): marca o início do quadro para sincronização
- TYPE: tipo — DATA(0), ACK(1), END(2), IMG(3)
- SEQ: número de sequência, alterna entre 0 e 1
- LEN: bytes úteis no DATA (0 a 16)
- DATA: dados transmitidos, sempre 16 bytes (zeros no restante)
- FCS: resultado do CRC-8 para detecção de erros

6. Detecção de Erros — CRC-8

O sistema utiliza CRC-8 com polinômio gerador 0x87 para detectar erros de transmissão. Mais robusto que checksum simples, detecta erros em rajada (burst errors).

6.1 Cálculo do FCS

O FCS é calculado sobre TYPE + SEQ + LEN + DATA (19 bytes). FLAG e FCS são excluídos:

```
crc = 0x00
para cada byte b nos bytes [1..19] do frame:
    crc = crc XOR b
    para j de 0 a 7:
        se (crc AND 0x80) != 0:
            crc = (crc << 1) XOR 0x87
        senao: crc = (crc << 1)
FCS = crc
```

6.2 Validação no Receptor

O receptor recalcula o CRC incluindo o FCS recebido. Se o resultado for zero, o quadro é aceito; caso contrário, é descartado:

```
se recalcular_crc(bytes[1..20]) == 0: aceitar
senao: descartar quadro
```

7. Controle de Fluxo — Stop-and-Wait ARQ

O protocolo Stop-and-Wait ARQ garante entrega confiável:

- Transmissor envia um quadro e para
- Aguarda ACK por até 5 segundos (timeout)
- Se receber ACK com SEQ correto, avança para o próximo quadro
- Se timeout, retransmite o mesmo quadro
- Tenta até 10 vezes antes de desistir

7.1 Preâmbulo

Antes de cada quadro, o transmissor envia 12 bytes do valor 0x2A em 4B6B para o receptor sincronizar o clock.

7.2 Turnaround

O receptor aguarda 500ms antes de enviar o ACK, permitindo ao transmissor chavear do modo TX para RX.

8. Tipos de Quadro

Tipo	Valor	Descrição	Quem envia
DATA	0	Mensagem de texto (até 16 chars)	Transmissor
ACK	1	Confirmação de recebimento	Receptor
END	2	Fim da transmissão	Transmissor
IMG	3	Imagem: 8 bytes do caractere 5x8px	Transmissor

9. Display LCD

O receptor utiliza LCD 16x2 com módulo I2C (0x27) nos pinos SDA=21 e SCL=22. Ao receber DATA, exibe o texto na linha 0. Ao receber IMG, usa `lcd.createChar()` para criar o caractere gráfico 5x8 pixels com os 8 bytes recebidos.

A imagem implementada representa um leão:

```
0x0E = 01110   0x0E = 01110   0x1F = 11111   0x1F = 11111
0x1F = 11111   0x0E = 01110   0x0A = 01010   0x0A = 01010
```

10. Simulação de Erros

O sistema demonstra detecção e recuperação de erros. Quando ocorre interferência RF, o CRC detecta a corrupção e o quadro é descartado. O transmissor retransmite automaticamente ao não receber ACK.

Log do monitor serial demonstrando o processo:

```
[TX] Enviado seq=0 tipo=0 len=14 (tentativa 1)
[RX] REJEITADO: FCS invalido (seq=0)
[TX] Timeout aguardando flag do ACK
[TX] Timeout — retransmitindo (tentativa 2)
[RX] ACEITO TEXTO [SEQ 0]: Ola Professor
[RX] ACK enviado seq=0
[TX] ACK confirmado seq=0
```

11. Resultados Obtidos

- Comunicação bem-sucedida via módulos RF 433 MHz
- Mensagens de texto exibidas corretamente no LCD
- Imagem (leão) reconstruída e exibida no LCD como caractere gráfico
- CRC-8 detectando e descartando quadros corrompidos
- Retransmissão automática funcionando após timeout
- Comunicação estável a distâncias superiores a 1 metro

12. Conclusão

O projeto foi implementado com sucesso, atingindo todos os objetivos propostos. O sistema demonstrou a aplicação prática de codificação de linha (4B6B), estruturação de quadros, detecção de erros (CRC-8) e controle de fluxo (Stop-and-Wait ARQ).

O CRC-8 mostrou-se eficaz na detecção de erros do canal RF. O Stop-and-Wait garantiu entrega confiável mesmo sob presença de ruído. O LCD exibiu corretamente mensagens de texto e imagens reconstruídas.

Referências

- Documentação oficial ESP32 — Espressif Systems:
<https://docs.espressif.com>
- Biblioteca LiquidCrystal_I2C:
https://github.com/johnrickman/LiquidCrystal_I2C